VIRAL PRODUCT INTELLIGENCE:
Early Detection and Predictive Modeling of Consumer Demand Surges Using Multi-Source Signals

Course: 23CSE452 Business Analytics (Units 1, 2, and 3 Final Submission)
Programme: B.Tech Computer Science and Engineering
Department: Computer Science and Engineering, Amrita School of Computing
Institution: Amrita Vishwa Vidyapeetham, Coimbatore, Tamil Nadu, India
Academic Year: 2026–2027 | Submission Date: November 2026

Project Team: Group 13

Mounik Sai (23CSE561) — Method 1 Forecasting, Method 2 Isolation & Viral Radar
Ashwin E (23CSE09) — Data Collection Pipeline, Text Corpus & Text Mining
Vijay Krishna (23CSE112) — Master Dataset, EDA & Text-Mining Evaluation
Sisthick S (23CSE465) — Feature Engineering, Scaling & Streamlit Dashboard
Konduru Hemesh (23CSE724) — Time-Series Preparation & Pipeline Consolidation

Project Guide: Mrs.Rohini S, Amrita School of Computing

Declaration

We, the members of Group 13, declare that this report, **"Viral Product Intelligence: Early Detection and Predictive Modeling of Consumer Demand Surges Using Multi-Source Signals"**, submitted for 23CSE452 Business Analytics, is our original work. The data were collected by the team from Google Trends and the YouTube Data API v3; no pre-built repository dataset was used. Sources are cited, generative-AI use is disclosed in Section 8.3, and the work has not been submitted for any other course or award.

Table 1. Declaration signatories

Name	Register No.	Signature
Mounik Sai	23CSE561	
Ashwin E	23CSE09	
Vijay Krishna	23CSE112	
Sisthick S	23CSE465	
Konduru Hemesh	23CSE724	

Place: Coimbatore Date: November 2026

Acknowledgement

We thank our project guide and the faculty of the Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for their guidance across Reviews 1 and 2, and the open-source communities behind pandas, scikit-learn, statsmodels, LightGBM, vaderSentiment and Streamlit.

Abstract

Retailers in India's smartphone market usually see viral demand only after it reaches sales data. This project tests whether public digital signals can flag demand surges earlier. We collected an original dataset for 60 smartphones (13 brands; 24 RISING, 24 STABLE, 12 DECLINING): 269 days of daily Google Trends search interest for India (8 Jan–3 Oct 2026) and YouTube Data API snapshots of 3,149 review videos and 29,189 unique comments, consolidated into a validated master dataset of 16,140 product-days × 43 variables. Unit 1 analysis shows strong short-lag persistence and cohort-specific dynamics. Unit 2 builds leak-free lag features and a next-day surge classifier; the selected Random Forest reaches a held-out PR-AUC of 0.222 versus 0.065 for chance. Unit 3 Method 1 (time-series forecasting) benchmarks ten models on a 53-day chronological test window; LightGBM reaches MAE 5.18 index points (R^2 0.736), 6.6% below naive persistence. Unit 3 Method 2 (text mining) applies VADER sentiment, LDA/NMF topics and lead-lag tests; comment activity moves with search in the same week but does not lead it. Results are delivered through a six-page Streamlit dashboard backed by a ten-stage orchestrated pipeline and 222 passing tests.

Keywords: demand sensing, Google Trends, YouTube, time-series forecasting, text mining, surge detection.

Table *2*. Abbreviations

Abbreviation	Meaning
ARIMA / ETS	Autoregressive integrated moving average / exponential smoothing
CV	Cross-validation
EDA	Exploratory data analysis
KPI	Key performance indicator
LDA / NMF	Latent Dirichlet allocation / non-negative matrix factorisation
MAE / RMSE / WAPE	Mean absolute / root mean squared / weighted absolute percentage error
MDA	Mean directional accuracy
PR-AUC / ROC-AUC	Area under the precision–recall / ROC curve
SI	Google Trends search interest (0–100)
VADER	Valence Aware Dictionary and sEntiment Reasoner

Executive Summary

Problem. Smartphone demand in India can spike within days of launches, reviews or price news. Planners who wait for sales reports react late: stock-outs on surging models, excess stock on fading ones.

What we built. A pipeline that tracks 60 phones daily using two free public signals—Google searches and YouTube reviews and comments—and turns them into forecasts, next-day surge alerts and a dashboard.

- **Data:** 16,140 validated product-days, no gaps or duplicates; 806 surge days (+50% day-on-day search jump).
- **Forecasts:** average error 5.2 points on a 0–100 scale—6.6% better than "tomorrow equals today" and 13.8% better than ARIMA.
- **Surge alerts:** 3.4× better than chance at ranking surge days; 330 alerts in 53 days catch 36% of surges.
- **Audience voice:** 58% of comments positive, 12% negative; comment tone does not predict next week's demand.

Business value. Both KPI targets were met. Under illustrative base-case costs (₹25,000 protected per anticipated surge-day, ₹4,000 per alert) alerts net ₹5.5 lakh over 53 days; they lose money if an alert costs more than ~23% of a surge-day's value, so thresholds must match each retailer's costs.

Actions. Use forecasts for short-horizon allocation; route RISING-model alerts to category managers as a daily check list; use sentiment for brand health, not demand; pilot with one retailer and add sales data.

Table of Contents

Declaration i
Acknowledgement i
Abstract ii
Executive Summary iii
1. Introduction, Business Objectives and Literature Review 1
1.1 Context and scope 1
1.2 Related work and research gap 2
2. Dataset Collection, Validation and Preparation 2
2.1 Provenance, coverage and challenges 2
2.2 Data quality, privacy and ethics 3
2.3 Transformations and usable dataset 3
3. Exploratory Data Analysis and Visualization (Unit 1) 3
4. Feature Engineering and Predictive Modelling (Unit 2) 5
5. Advanced Analytics (Unit 3): Two Selected Methods 6
5.1 Method 1 — Time-series forecasting of daily search interest 6
5.2 Method 2 — Text mining of YouTube comments 7
6. Comparative Analysis and Business Impact 9
6.1 Comparison of the two Unit 3 methods 9
6.2 Comparison with existing studies 9
6.3 KPI-based business impact and scenarios 10
7. Interactive Application, Figure Index and Reproducibility 10
7.1 Dashboard 10
7.2 Figure index and reproduction map 12
7.3 Code and environment 12
8. End-to-End Validation, Traceability and Contributions 13
8.1 Completion and validity 13
8.2 Project tracking and individual work 13
8.3 AI use and originality declarations 14
9. Conclusion, Recommendations and Roadmap 14
References 14
Appendices 16
Appendix A: Validation records 16
Appendix B: Model tuning 16
Appendix C: Implementation and dashboard guide 17
Appendix D: Tracking, AI declaration and similarity report 17

List of Figures and Tables

Figure 1. End-to-end workflow linking Units 1–3 and the application. Source: reports/make_report_figures.py 1
Figure 2. Mean daily search interest by cohort (7-day moving averages). Source: notebooks/04_eda_full.ipynb 4
Figure 3. Autocorrelation of search interest and its correlation with the surge label, averaged over 60 products. Source: src/prep_time_series.py 4
Figure 4. Precision–recall curves on the test window (left) and permutation importance of the Random Forest (right). Source: reports/unit2_surge_classification.py 5
Figure 5. Forecast versus actual search interest for representative products in the test window. Source: src/forecast_method1.py 7

Figure 6. Top-10 terms of the three LDA topics (top) and NMF comparison (bottom). Source: reports/make_report_figures.py from data/processed/text_topics_terms.csv 8
Figure 7. Weekly lead-lag correlation of comment signals with search change (* $p < 0.05$). Source: src/evaluate_text_mining.py 8
Figure 8. Net impact versus alert threshold for the three scenarios. Source: reports/make_report_figures.py 10
Figure 9. Overview page filtered to Samsung / RISING (4 products, 33 surge days); every KPI and chart updates. Source: running Streamlit app 11
Figure 10. Viral Radar page: demand acceleration versus sentiment with breakout-score KPIs. Source: running Streamlit app 12

Table 1. Declaration signatories i
Table 2. Abbreviations ii
Table 3. Business objectives, KPIs and success criteria 1
Table 4. Data sources (public data under Google's terms; team-collected) 2
Table 5. Sample rows of the final dataset (selected columns) 2
Table 6. Mapping of final fields to Units 1–3 3
Table 7. Descriptive statistics by cohort 3
Table 8. Unit 2 surge classification on the held-out window (3,180 rows, 208 surges) 5
Table 9. Method 1 benchmark on the 53-day test window (3,180 product-days) 6
Table 10. Sentiment validation against reference labels 8
Table 11. Comparison of the two Unit 3 methods 9
Table 12. Comparison with prior studies 9
Table 13. Best/base/worst-case net impact over the 53-day test window 10
Table 14. Figure index (all figures created in Python/Matplotlib or captured from the app) 12
Table 15. Objective-to-evidence traceability matrix 13
Table 16. Individual contributions and evidence 13
Table 17. Prioritised recommendations 14
Table 18. Master dataset validation (processed/master_validation_report.csv) 16
Table 19. Method 2 leakage audit (processed/method2_leakage_audit_report.csv) 16
Table 20. Unit 2 expanding-window CV grid (5 date folds) 16
Table 21. Pipeline execution record (processed/pipeline_execution_summary.csv) 17

1. Introduction, Business Objectives and Literature Review

1.1 Context and scope

India's smartphone market has frequent launches, short life cycles and strong reviewer influence. Demand for a model can surge within days and fade when a successor arrives. Retail category managers, brand marketers and distributors all need earlier warning than sales data give. We use public *leading* signals—Google search interest (purchase intent) and YouTube reviews and comments (awareness and opinion)—for 60 phones sold in India, 8 Jan–3 Oct 2026. Unit sales and prices are out of scope (no public, terms-compliant source), so search interest serves as the demand proxy. Questions: **Q1** how signals differ by cohort and brand; **Q2** can next-day surges be flagged; **Q3** how accurately can next-day interest be forecast; **Q4** what audiences say and whether it leads demand; **Q5** can results be delivered as a usable tool.

Table *3*. Business objectives, KPIs and success criteria

Objective	KPI / unit	Baseline / target	Unit
O1 Validated daily dataset	Product-day coverage; checks passed	Target 100% of 60×269; 0 failures	U1
O2 Flag next-day surges	PR-AUC (held-out)	Chance 0.065 / target ≥ 2× chance	U2
O3 Forecast next-day interest	MAE (index points)	Naive 5.54 / target ≥ 5% lower	U3 M1
O4 Measure opinion and lead time	Accuracy vs reference labels; lead-lag ρ	Always-positive 0.398 / beat it; test positive lag	U3 M2
O5 Decision support	Working pages and filters; tests	All units in one app; 100% tests pass	App

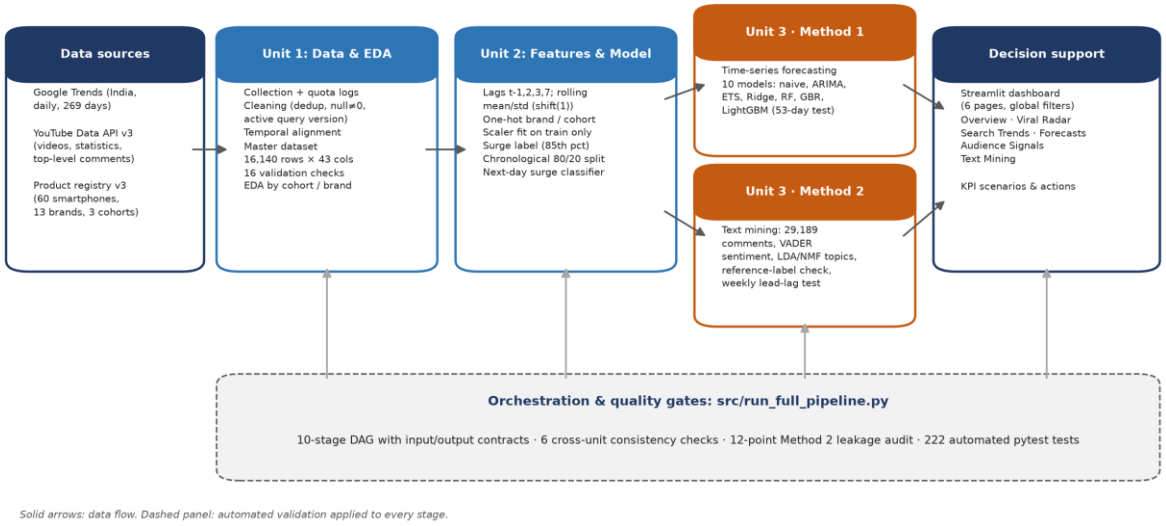


Figure *1*. End-to-end workflow linking Units 1–3 and the application. *Source: reports/make_report_figures.py*

1.2 Related work and research gap

Choi and Varian [1] showed that adding Google Trends to seasonal autoregressive models improves nowcasts of US retail and travel series by roughly 5–20%, using aggregated weekly or monthly data. Asur and Huberman [2] predicted box-office revenue from tweet rates (adjusted $R^2 \approx 0.80$), a single-launch setting. Boone et al. [3] reviewed consumer analytics in sales forecasting and found benefits mainly for new products and short horizons. Carbonneau et al. [4] found machine-learning demand forecasts best but not significantly better than regression. VADER [5] reached F1 ≈ 0.96 on English tweets. **Gap:** prior work uses aggregates, one signal at a time, English-only text and rarely documents leakage controls. We therefore build a daily, product-level, multi-source panel for Indian phones, evaluate chronologically against naive baselines, validate sentiment on local code-mixed comments and audit leakage.

2. Dataset Collection, Validation and Preparation

2.1 Provenance, coverage and challenges

Table 4. Data sources (public data under Google's terms; team-collected)

Source	Method	Dates	Volume
Google Trends (India, web)	pytrends with polite pauses/back-off; loader validates query, range, daily granularity	8 Jan–3 Oct 2026	60 series, 16,140 rows
YouTube Data API v3	search (top 50, region IN), video statistics, top 100 comments on 5 most-viewed videos	Snapshots 3–4 Oct 2026	3,149 videos; 29,189 comments
Product registry v3	Team-curated: queries, brand, launch period, cohort	3 Oct 2026	60 phones, 13 brands

The master dataset `processed/master_dataset.csv` has **16,140 rows × 43 variables**, one row per product-day. Targets: `surge_label` (1 if day-on-day growth ≥ +50%, the 85th percentile of positive growth) for Unit 2 and `search_interest` for Method 1.

Table 5. Sample rows of the final dataset (selected columns)

ID	Date	Brand	Cohort	SI	Growth	Videos	Comments	Surge
P045	2026-04-03	Oppo	STABLE	38	+0.73	0	1	1
P008	2026-08-08	Google	RISING	14	-0.07	0	0	0
P048	2026-08-21	Poco	STABLE	68	+0.00	0	0	0
P042	2026-08-22	Realme	STABLE	15	+0.50	0	2	1
P045	2026-08-30	Oppo	STABLE	39	+0.44	0	0	0
P059	2026-09-11	Redmi	DECLINING	51	+0.65	0	0	1

Challenges. YouTube quotas (10,000 units, 100 searches/day) were handled by a quota tracker, caching and a measured cost of 7 units per product; Trends rate limits by back-off; per-product Trends scaling by using within-product changes; sibling-model results (e.g., “Pro” variants) by an

on-target title matcher that flags products below 60%; and snapshot timing by rebuilding daily activity from publish timestamps.

2.2 Data quality, privacy and ethics

The builder writes the master file only if all checks pass (14 PASS, 0 FAIL): 60 products × 269 days with no gaps; 0 duplicate keys; interest within 0–100; no negative counts; growth left missing on 3,688 undefined rows (never set to 0); 15 low-signal and 4 low on-target products flagged, not dropped; 1,100 rows from superseded queries excluded with logged reasons; surge class imbalance 4.99%. **Privacy:** only public content is used; commenter names and channel IDs are stripped, raw API files are git-ignored, the API key stays in a local `.env`, and use is limited to this research purpose, consistent with India's DPDP Act, 2023 [12]. **Biases:** relevance-ranked, English-leaning YouTube samples; October comment snapshots (survival bias); Trends sampling and truncation; more Samsung/Apple models.

2.3 Transformations and usable dataset

General cleaning kept active query versions (comments 33,217 → 32,215), standardised timestamps, removed duplicates (→ 29,189 unique comments), kept outliers and flagged rather than dropped short, emoji-only and non-Latin comments (22,568 usable). Trends and YouTube were left-joined on the daily index without forward fill. **Model-specific** steps after the chronological split (cut-off 12 Aug 2026): lags t−1, 2, 3, 7; 7-day rolling mean/SD with `shift(1)`; one-hot brand/cohort; scaler and topic models fitted on training data only. After a 7-day burn-in: 12,540 train and 3,180 test rows.

Table 6. Mapping of final fields to Units 1–3

Field group	Meaning / preparation	Used in
search_interest; growth; accel	Daily Trends index and its changes	U1, U2, M1 target
surge_label	0/1, growth ≥ +50% (identical with a train-only cut-off)	U2 target
_lag_1/2/3/7, rolling_mean/std_7	Lagged search, growth, accel, videos, comments	U2, M1
videos_published, comments_published	Daily counts from YouTube publish timestamps	U1, U2, M1, M2
brand_ *, product_type_*	One-hot dummies	U1, U2, M1
text_clean, vader_compound, topic shares	Cleaned comments, sentiment, LDA/NMF topics	M2
usable_product, low_on_target_share	Quality flags	U1 bias checks

3. Exploratory Data Analysis and Visualization (Unit 1)

Table 7. Descriptive statistics by cohort

Cohort	Products	Mean SI	Median SI	Zero days	Surge rate	Comments
RISING	24	6.1	1	49.1%	6.2%	11728
STABLE	24	25.8	21	6.4%	4.5%	8045

Cohort	Products	Mean SI	Median SI	Zero days	Surge rate	Comments
DECLINING	12	32.7	29	1.4%	3.5%	1055

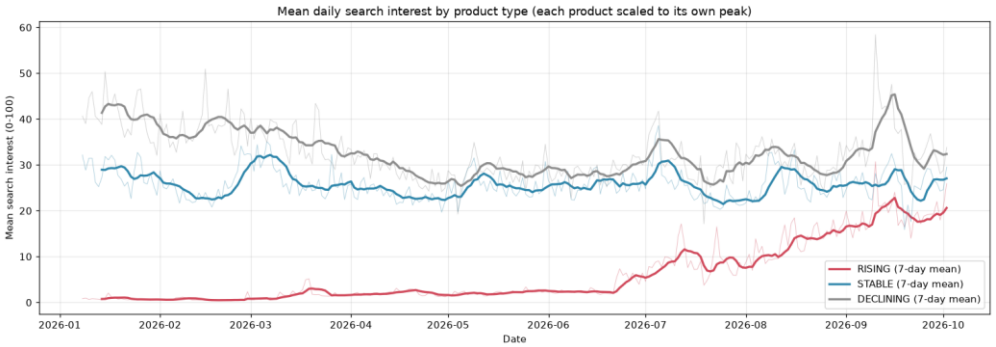


Figure *2*. Mean daily search interest by cohort (7-day moving averages). *Source: notebooks/04_eda_full.ipynb*

Interpretation. RISING models sit near zero until launch season and climb from late June (49% zero days); STABLE and DECLINING models fluctuate around 25–35 with short spikes, e.g., the mid-September flagship launches. Cohort therefore drives surge rates and forecast difficulty.

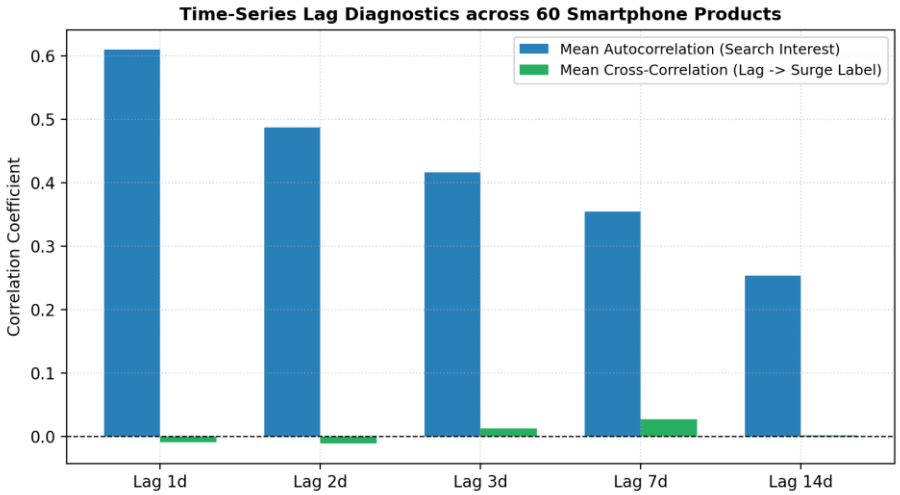


Figure *3*. Autocorrelation of search interest and its correlation with the surge label, averaged over 60 products. *Source: src/prep_time_series.py*

Interpretation. Interest is highly persistent (autocorrelation 0.61 at 1 day, 0.25 at 14 days), so naive persistence is a strong baseline; yet lagged *levels* barely correlate with surges ($|r| < 0.03$). This motivated short lags, rolling volatility and non-linear models.

Other findings: interest correlates only weakly with comments (Spearman 0.20) and videos (0.10), so social signals are complementary; weekly seasonality is mild (Monday 18.4 to Sunday 20.8); 95% of series are stationary after differencing, supporting ARIMA(1,1,1) as a benchmark.

Hypotheses carried forward: recent momentum predicts level and surges; cohort shifts baselines; comments may lead search.

4. Feature Engineering and Predictive Modelling (Unit 2)

Task. Using information up to day t−1, predict whether day t is a surge. **Features (36):** 20 lags, 2 leak-free rolling statistics and 14 brand/cohort dummies; no dimensionality reduction (small, interpretable set). **Protocol:** chronological split (train 586 surges, 4.67%; test 208, 6.54%); 5-fold expanding-window CV by date on training data to choose settings by PR-AUC; decision threshold set to maximise out-of-fold F1; test scored once. PR-AUC is primary because surges are rare [15].

Table *8*. Unit 2 surge classification on the held-out window (3,180 rows, 208 surges)

Model	Settings	CV PR-AUC	Precision	Recall	F1	ROC-AUC	PR-AUC
Persistence rule (baseline)	growth_lag_1 rule	–	0.055	0.495	0.099	0.431	0.078
Logistic Regression (baseline)	L2, C=0.1, balanced	0.095	0.120	0.211	0.153	0.616	0.099
Random Forest	300 trees, depth 10	0.231	0.227	0.361	0.279	0.751	0.222
HistGradientBoosting	lr 0.05, depth 4	0.208	0.168	0.471	0.248	0.751	0.235

Note: Chance PR-AUC = 0.065. Script: reports/unit2_surge_classification.py.

Random Forest had the best CV PR-AUC (0.231) and was fixed before testing. On test it reaches PR-AUC 0.222 (3.4× chance) and ROC-AUC 0.751, double logistic regression (0.099), confirming non-linear structure. It catches 75 of 208 surges with 255 false alarms and 133 misses; precision ≈ 23% makes it a triage tool, not an auto-reorder rule. **Leakage audit:** all features lagged; rolling windows exclude day t; scaler fit on train; split by date; the surge cut-off recomputed on training data alone is identical (all 16,140 labels unchanged); Unit 1/Unit 2 label parity 100%.

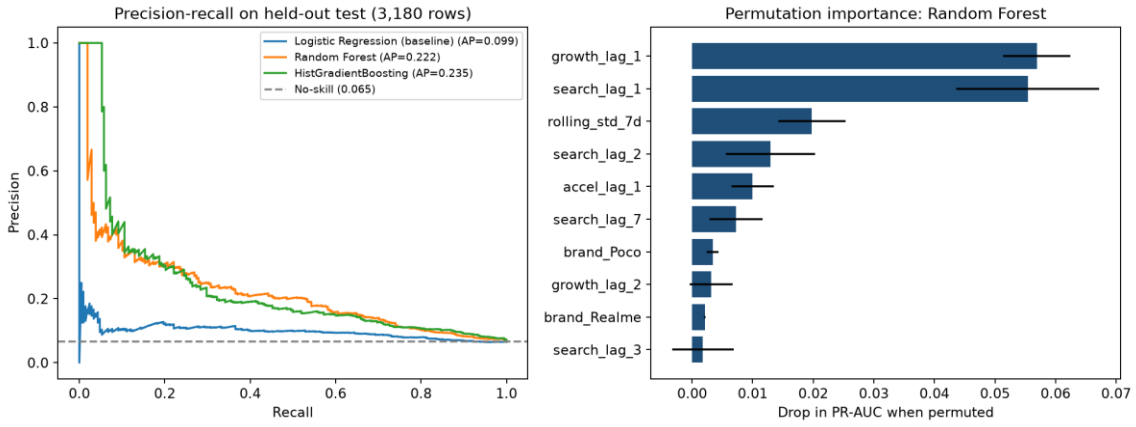


Figure *4*. Precision–recall curves on the test window (left) and permutation importance of the Random Forest (right). *Source: reports/unit2_surge_classification.py*

Interpretation. Tree ensembles dominate logistic regression at all recalls. Yesterday's growth (0.057 PR-AUC drop) and level (0.055), then 7-day volatility, matter most; YouTube lags add little. Importances are associations, not causes.

Subgroup check. ROC-AUC is similar for RISING (0.76) and STABLE (0.74) but lower for DECLINING (0.67), where rarer surges follow successor launches the lags cannot see; recall is highest for RISING (40%), where early warning matters most. Cohort-specific thresholds and launch-date features are the proposed mitigation.

5. Advanced Analytics (Unit 3): Two Selected Methods

We selected **Time-Series Analysis** and **Text Mining**; association rules do not apply (no baskets).

5.1 Method 1 — Time-series forecasting of daily search interest

Question (Q3): how accurately can tomorrow's interest be predicted for allocation decisions? **Input:** 12,540 train and 3,180 test product-days (12 Aug–3 Oct, 53 days). **Models:** naive ($\hat{y}_t = y_{t-1}$), seasonal naive (y_{t-7}), 7-day mean; per-product ARIMA(1,1,1) and Holt smoothing; Ridge, ElasticNet, Random Forest, Gradient Boosting and LightGBM [7] (100 trees, depth 4, learning rate 0.05) on the 36 features; one-step forecasts clipped to 0–100; metrics per [14]. Code: `src/prep_time_series.py`, `src/forecast_method1.py`, `notebooks/scrum26_time_series_forecasting.ipynb`.

Table 9. Method 1 benchmark on the 53-day test window (3,180 product-days)

Model	Family	MAE	RMSE	WAPE %	R^2	MDA %
LightGBM	Machine Learning	5.176	10.027	21.52	0.736	50.3
Gradient Boosting	Machine Learning	5.288	10.200	21.99	0.726	49.6
Random Forest	Machine Learning	5.294	10.108	22.01	0.731	49.3
Ridge Regression	Machine Learning	5.304	10.349	22.05	0.718	50.9
ElasticNet	Machine Learning	5.363	10.406	22.30	0.715	50.6
Naive Persistence	Baseline	5.544	11.204	23.05	0.670	50.0
Exponential Smoothing	Statistical	5.601	11.291	23.29	0.665	50.1
ARIMA(1,1,1)	Statistical	6.002	11.763	24.95	0.636	51.6
Rolling Mean 7d	Baseline	6.293	11.839	26.17	0.631	51.5
Seasonal Naive 7d	Baseline	8.995	16.783	37.40	0.259	53.0

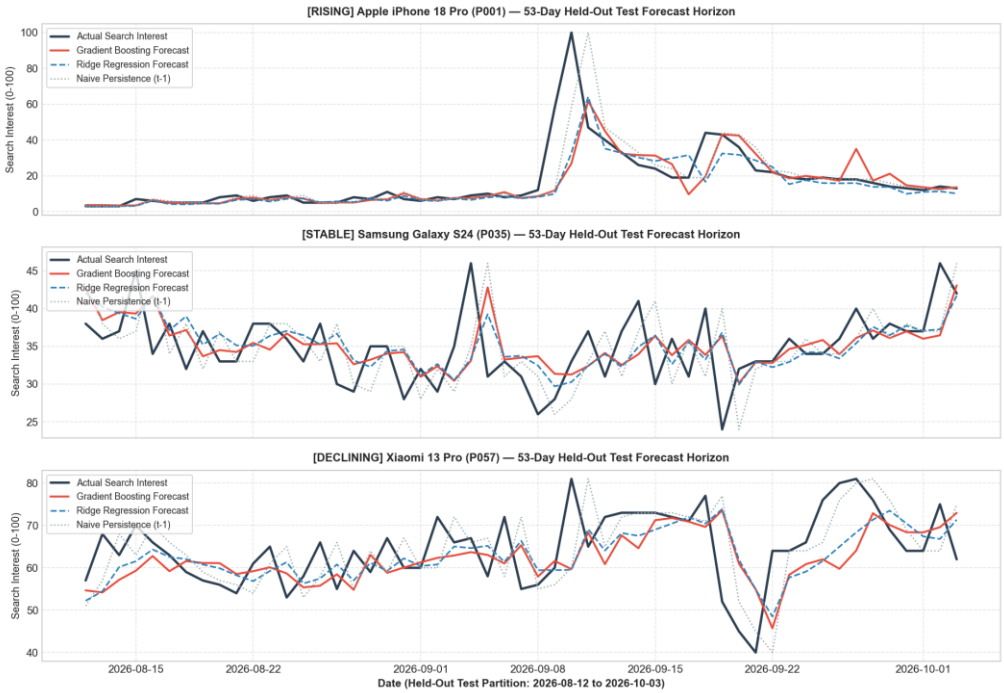


Figure *5*. Forecast versus actual search interest for representative products in the test window. *Source: src/forecast_method1.py*

Interpretation. Forecasts follow level shifts within a day but lag the first day of sharp spikes—the reason a separate surge classifier is needed.

Findings. All ML models beat every baseline; LightGBM is best: MAE 5.176 (6.6% below naive, 13.8% below ARIMA) and RMSE 10.5% lower than naive. Gains are largest for STABLE (11.2%, R^2 0.79) and DECLINING (8.1%) and small for RISING (1.9%). `search_lag_1` and the 7-day mean carry ~95% of tree importance. **Limitations:** MDA ≈ 50% for all models (no directional skill); fixed hyperparameters; one 53-day window; without LightGBM installed the script substitutes Extra Trees.

5.2 Method 2 — Text mining of YouTube comments

Question (Q4): what do audiences say, and does comment activity or tone lead search? **Input:** 29,189 comment rows (22,568 usable). **Steps:** cleaning, domain stop-list (platform words, Hinglish fillers) and removal of brand tokens; VADER [5] compound score (positive ≥ 0.05, negative ≤ −0.05); LDA [6] and NMF [11] for k = 3–10 chosen by NPMI coherence and refit stability; validation on 100 random comments (5 human, 95 AI-review labels, flagged per row) against TextBlob and an always-positive baseline; weekly Spearman lead-lag of comment and

sentiment changes vs search change. Code: `src/text_mining.py`, `src/evaluate_text_mining.py`, `notebooks/05_text_mining.ipynb`.

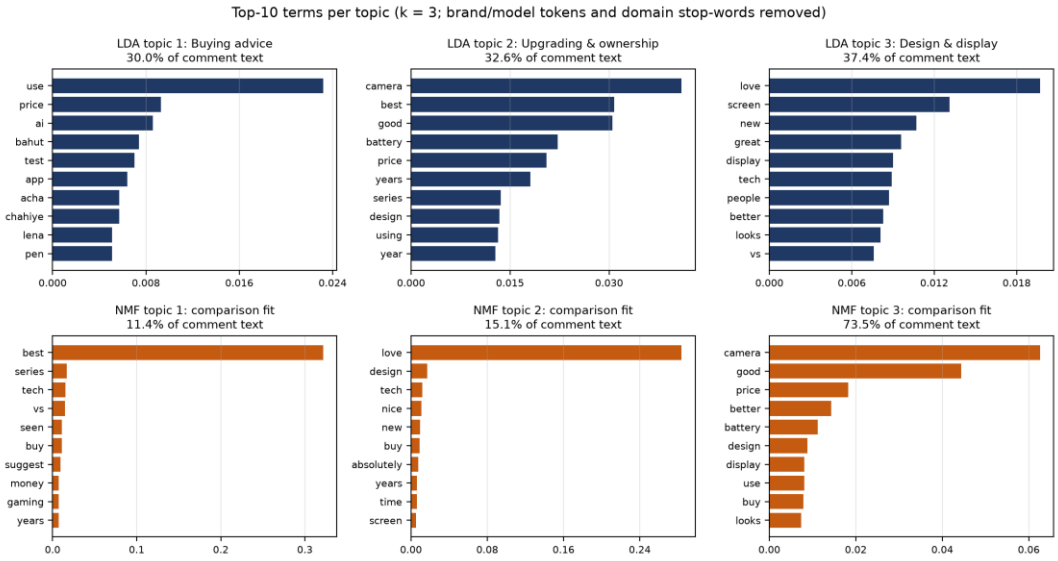


Figure *6*. Top-10 terms of the three LDA topics (top) and NMF comparison (bottom). *Source: reports/make_report_figures.py from data/processed/text_topics_terms.csv*

Interpretation. Topics: *Buying advice* (price, AI, Hinglish "chahiye", "lena"; 30%), *Upgrading & ownership* (camera, battery, years; 33%) and *Design & display* (love, screen, looks; 37%, 45% for DECLINING models). NMF recovers the camera/price and design themes. Coherence is low (LDA 0.021, NMF 0.085) because comments are short (median 5 tokens).

Table *10*. Sentiment validation against reference labels

Method (93 labelled comments)	Accuracy [95% CI]	Macro-F1	Neg. recall	κ
VADER (project)	0.495 [0.40–0.59]	0.468	0.267	0.22
TextBlob	0.419 [0.32–0.52]	0.375	0.100	0.13
AI review (SCRUM-28 spot check)	0.796 [0.70–0.87]	0.828	0.700	0.70
Always positive (baseline)	0.398 [0.30–0.50]	0.190	0.000	0.00

Note: The AI-review row is not independent: it supplied most reference labels.

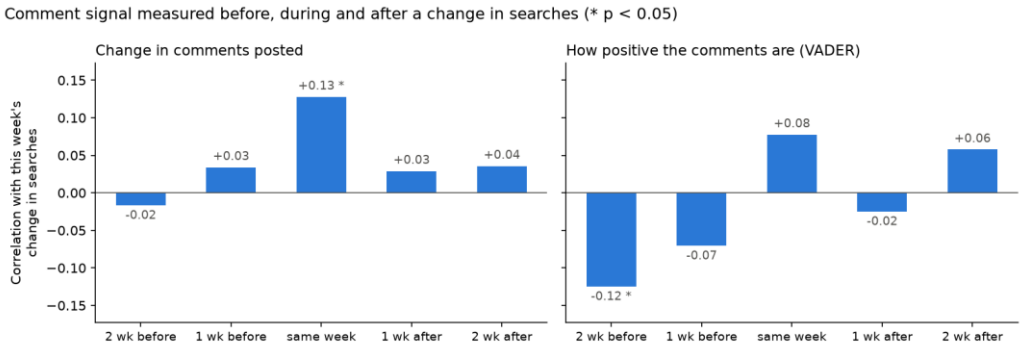


Figure *7*. Weekly lead-lag correlation of comment signals with search change (* p < 0.05). *Source: src/evaluate_text_mining.py*

Interpretation. Comment volume rises in the *same* week as search ($\rho = 0.13$, $p < 0.001$) but not a week earlier ($\rho = 0.03$, $p = 0.12$); sentiment two weeks earlier is weakly *negative* ($\rho = -0.12$, $p = 0.006$). Comments are a coincident, not leading, signal.

Findings. 58.4% of scored comments are positive, 11.5% negative; Oppo (0.39) and iQOO/Realme (0.34) audiences are most positive, Motorola and Nothing least (0.27). VADER beats TextBlob and the baseline (accuracy 0.495 vs 0.419 and 0.398) but misses most negatives (recall 0.27) due to Hinglish, sarcasm and jargon. Method 2 model inputs passed 11 of 12 leakage checks (one informational note on October-snapshot survival bias).

6. Comparative Analysis and Business Impact

6.1 Comparison of the two Unit 3 methods

Table *11*. Comparison of the two Unit 3 methods

Criterion	Method 1: Time-series	Method 2: Text mining
Question / input	Next-day level; numeric daily panel	Opinion and lead time; 29,189 comments
Output / validation	Point forecasts; chronological hold-out vs 9 alternatives	Sentiment, topics, ρ; reference labels, coherence, significance
Strengths	Clear gains over naive and ARIMA; reproducible	Explains *why*; brand-level insight
Limitations	No directional skill; lags spike onsets	English lexicon; short text; not leading

The metrics are not comparable (MAE vs accuracy), so neither method is "better". **Synthesis:** Units 1, 2 and Method 1 agree that recent momentum dominates (high autocorrelation, top importance of lagged growth and level); YouTube activity is consistently secondary and, per Method 2, coincident. The Review 1 hypothesis that sentiment leads demand by 3–7 days is not supported, so text serves as a brand-health layer while numeric momentum drives alerts. The dashboard's Viral Radar combines both as a descriptive 0–100 score (weights 0.40 level, 0.35 acceleration, 0.25 sentiment), not a validated predictor.

6.2 Comparison with existing studies

Table *12*. Comparison with prior studies

Criterion	Choi & Varian [1]	Asur & Huberman [2]	Hutto & Gilbert [5]	This project
Data	US retail/travel; weekly–monthly Trends	Box office; tweet rate	English tweets	60 Indian phones; daily Trends + YouTube
Validation	Rolling out-of-sample	In-sample R^2	F1 vs human labels	Chronological hold-out, CV, leakage audit
Finding	Error −5 to −20% vs AR	Adj. $R^2 \approx 0.80$	F1 ≈ 0.96	MAE −6.6% vs naive, −13.8% vs ARIMA; VADER F1 0.47

Direct comparison is limited: Choi and Varian measure the gain of *adding* Trends to forecast sales, while our target is the Trends index; Asur and Huberman report in-sample R^2 for single launches; VADER's 0.96 is binary English sentiment, whereas our 0.47 is three-class, on Hinglish-heavy comments with mostly AI reference labels—largely domain shift. Our contribution is a reproducible daily product-level panel for an emerging market, explicit leakage controls, cohort diagnostics and an honest negative lead-lag result.

6.3 KPI-based business impact and scenarios

Measured: forecast MAE −6.6% (O3 met); surge PR-AUC 3.4× chance with 75 true alerts of 330 (O2 met). **Hypothetical economics:** net = true alerts × value per anticipated surge-day − alerts × cost per alert.

Table *13*. Best/base/worst-case net impact over the 53-day test window

Scenario	Value / surge-day (₹)	Cost / alert (₹)	Benefit (₹ lakh)	Cost (₹ lakh)	Net (₹ lakh)
Worst	10,000	6,000	7.50	19.80	**-12.30**
Base	25,000	4,000	18.75	13.20	**+5.55**
Best	50,000	2,000	37.50	6.60	**+30.90**

Note: Illustrative assumptions; replace with partner data.

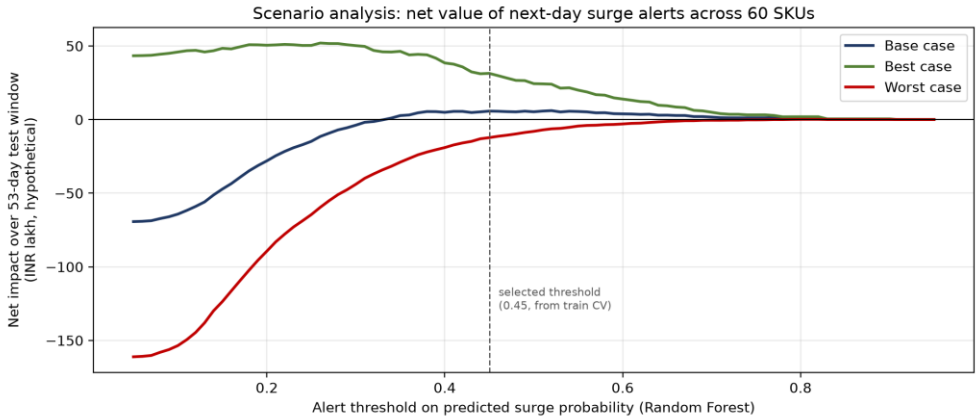


Figure *8*. Net impact versus alert threshold for the three scenarios. *Source: reports/make_report_figures.py*

Interpretation. Base case nets ₹5.5 lakh and stays positive above a threshold of ~0.33; the worst case loses at almost every threshold. Break-even needs surge-day value > 4.4× alert cost; the key dependencies are stock-out margin, action cost and search-to-sales conversion.

7. Interactive Application, Figure Index and Reproducibility

7.1 Dashboard

A local multi-page Streamlit [13] app: `pip install -r requirements.txt`, then `streamlit run dashboard/app.py` (or `python -m streamlit run dashboard/app.py`). `data_loader.py` caches

the committed outputs; sidebar filters (category, brand, cohort, products, date range, reset) persist across pages via `filters.py`. Pages: Overview (U1 KPIs), Viral Radar (U2+U3 synthesis), Search Trends (surge markers, calendar), Forecasts (M1, errors recomputed per selection), Audience Signals and Text Mining (M2). Filtering, KPIs and the radar score are computed live; models and scores are precomputed.

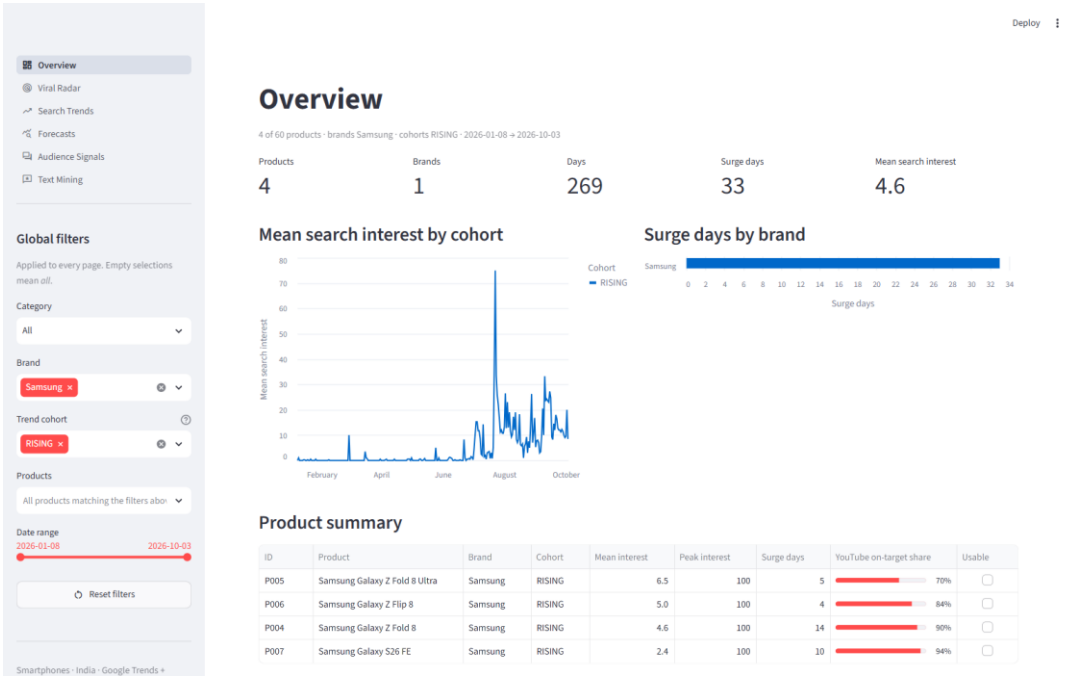


Figure 9. Overview page filtered to Samsung / RISING (4 products, 33 surge days); every KPI and chart updates.
Source: running Streamlit app

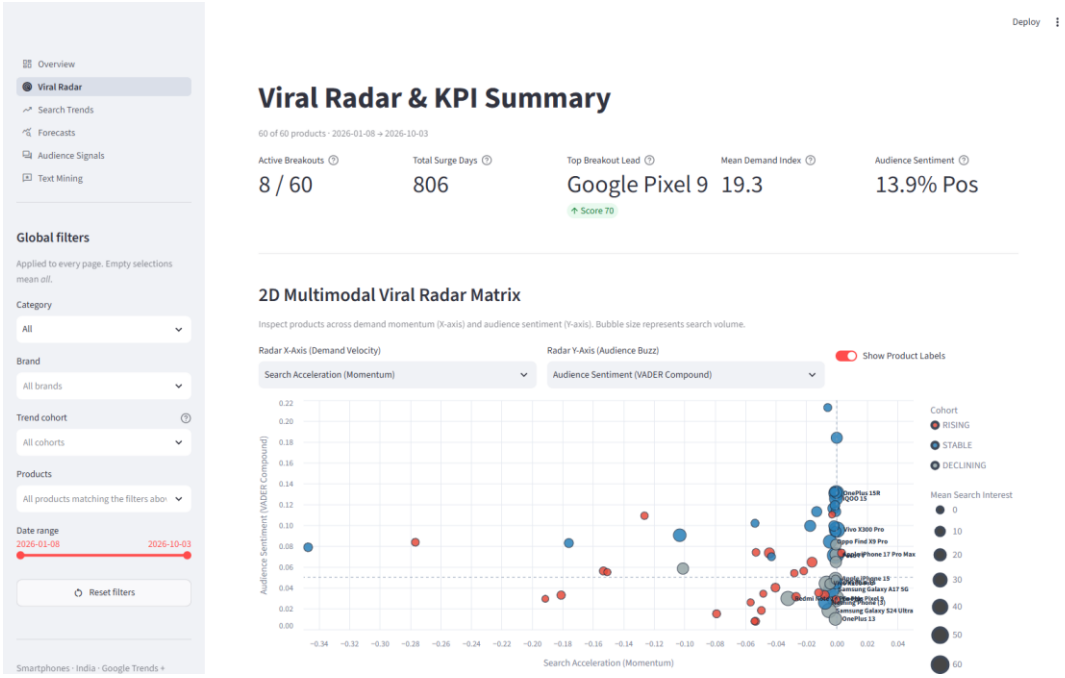


Figure *10*. Viral Radar page: demand acceleration versus sentiment with breakout-score KPIs. *Source: running Streamlit app*

Tests and limitations. `tests/test_dashboard.py` covers filter logic and headless page renders (all pass). Forecast headline cards show the global benchmark; radar weights are not back-tested; the app is not publicly hosted; data end on 3 Oct 2026.

7.2 Figure index and reproduction map

Table *14*. Figure index (all figures created in Python/Matplotlib or captured from the app)

Fig.	Title	Section	Source file
1	End-to-end workflow linking Units 1–3 and the application	1	reports/make_report_figures.py
2	Mean daily search interest by cohort (7-day moving averages)	3	notebooks/04_eda_full.ipynb
3	Autocorrelation of search interest and its correlation with the surge label	3	src/prep_time_series.py
4	Precision–recall curves on the test window (left) and permutation importanc	4	reports/unit2_surge_classification.py
5	Forecast versus actual search interest for representative products in the t	5	src/forecast_method1.py
6	Top-10 terms of the three LDA topics (top) and NMF comparison (bottom)	5	reports/make_report_figures.py from data/processed/text
7	Weekly lead-lag correlation of comment signals with search change (* p < 0.	5	src/evaluate_text_mining.py
8	Net impact versus alert threshold for the three scenarios	6	reports/make_report_figures.py
9	Overview page filtered to Samsung / RISING (4 products, 33 surge days); eve	7	running Streamlit app
10	Viral Radar page: demand acceleration versus sentiment with breakout-score	7	running Streamlit app

7.3 Code and environment

Repository: https://github.com/Ashwin-deals/viral-product-intelligence- (public README with tree and commands). Python 3.12; dependencies in `requirements.txt`. Layout: `raw/`,

`data/processed/`, `processed/`, `src/`, `notebooks/`, `docs/figures/`, `dashboard/`, `tests/`, `reports/`. Reproduce: `python -m pytest`; `python src/run_full_pipeline.py` (Units 1–3); `python reports/unit2_surge_classification.py`. Raw YouTube JSON (with commenter names) and the API key are excluded; both sources can be re-collected with a personal key under Google's terms.

8. End-to-End Validation, Traceability and Contributions

8.1 Completion and validity

The orchestrator ran all 10 stages successfully (152 s) and passed six cross-unit checks; the test suite was re-run for this report: **222 passed, 0 failed**. **Threats:** the surge label derives from the input series (mitigated by lagging and a train-only cut-off check); AI reference labels; one country, category and season; search is a proxy for sales; Trends re-sampling.

Table *15*. Objective-to-evidence traceability matrix

Objective	Method	Result	Evidence	Action
O1 Dataset	Sec. 2	16,140/16,140 rows; 14 PASS	Sec. 2.2; Appendix A	R5
O2 Surge alerts	Sec. 4, RF	PR-AUC 0.222 = 3.4× chance	Table 8, Fig. 4	R2
O3 Forecast	Sec. 5.1, LightGBM	MAE −6.6% vs naive	Table 9, Fig. 5	R1
O4 Opinion & lead	Sec. 5.2	Acc. 0.495 > 0.398; no lead	Table 10, Figs. 6, 7	R3
O5 Decision tool	Sec. 7	6 pages, filters; 222 tests pass	Figs. 9–10	R4

8.2 Project tracking and individual work

Work was tracked on a Jira Scrum board (key SCRUM; Backlog → To Do → In Progress → Review/Testing → Completed; link: **[insert Jira board URL]**) and merged via GitHub pull requests. Commit counts are from `git shortlog`; individual marks may vary with participation and explanation.

Table *16*. Individual contributions and evidence

Member / Reg. No.	Work items (completed)	Evidence
Mounik Sai (23CSE561)	SCRUM-10, 14, 19, 24, 26, 32: documentation, pipeline consolidation, Method 2 isolation & audit, Method 1 benchmark, Viral Radar, presentations (4–21 Oct)	forecast_method1.py, isolate_method2_data.py, radar.py; 38 commits
Ashwin E (23CSE09)	Task 1, SCRUM-23, 28, 33: collection, cleaning, alignment, EDA, text corpus and mining, Text Mining page (4–21 Oct)	collect_youtube.py, text_mining.py; 12 commits
Vijay Krishna (23CSE112)	SCRUM-5, 12, 17, 21, 29: registry analysis, problem statement, cleaning docs, master dataset, text-mining evaluation (4–21 Oct)	build_master.py, evaluate_text_mining.py; 29 commits
Sisthick S (23CSE465)	SCRUM-20, dashboard: set-up, encoding & scaling, Streamlit app, time-series evaluation (3–21 Oct)	feature_encoding_scaling.py, dashboard/app.py; 30 commits
Konduru Hemesh (23CSE724)	SCRUM-22, 30: chronological split & stationarity, end-to-end orchestration and tests (20 Oct)	prep_time_series.py, run_full_pipeline.py; 5 commits

8.3 AI use and originality declarations

Anthropic Claude (via Claude Code and GitHub Copilot) assisted with code, documentation and drafting this report; the team verified every number against committed outputs, re-ran tests and corrected unsupported earlier slide claims (surge-classifier F1, sentiment lead time). An AI reviewer supplied 95 of 100 sentiment reference labels (flagged per row). Turnitin similarity: **[insert %]** (limit ≤ 15%).

9. Conclusion, Recommendations and Roadmap

All objectives were met (traceability matrix, Section 8.1): forecasts improve on naive by 6.6% and surge alerts rank 3.4× better than chance; comment tone is not a leading indicator and forecasts lack directional skill, which shapes how the tool should be used.

Table *17*. Prioritised recommendations

#	Recommendation	KPI effect	Owner	Feasibility
R1	Daily forecasts for STABLE/DECLINING allocation	MAE −8 to −11% vs naive	Planners	High
R2	RISING surge alerts as a daily triage list; threshold from cost ratio	~40% of surges a day early	Category managers	Medium
R3	Sentiment/topics as brand-health monitor	Early negative themes	Marketing	High
R4	One-season retail pilot of the dashboard	Measured stock-out change	Team + partner	Medium
R5	Add sales and price data	KPIs in units and ₹	Partner data team	Low–Medium

Roadmap. *0–1 month:* schedule daily collection and nightly pipeline runs (success: 30 failure-free days). *1–6 months:* partner pilot with cohort thresholds and launch-date features (success: alert precision ≥ 30%). *6–12 months:* sales data, Hinglish-aware sentiment, more categories. **Future work (not done):** interval and multi-day forecasts, Method 1 tuning, radar back-testing, larger human labelling, public hosting.

References

[1] H. Choi and H. Varian. 2012. Predicting the Present with Google Trends. *Economic Record* 88, s1, 2–9.

[2] S. Asur and B. A. Huberman. 2010. Predicting the Future with Social Media. In *Proc. IEEE/WIC/ACM WI-IAT*, 492–499.

[3] T. Boone, R. Ganeshan, A. Jain, and N. R. Sanders. 2019. Forecasting sales in the supply chain: Consumer analytics in the big data era. *International Journal of Forecasting* 35, 1, 170–180.

[4] R. Carbonneau, K. Laframboise, and R. Vahidov. 2008. Application of machine learning techniques for supply chain demand forecasting. *European Journal of Operational Research* 184, 3, 1140–1154.

[5] C. J. Hutto and E. Gilbert. 2014. VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. In *Proc. ICWSM*, 216–225.

[6] D. M. Blei, A. Y. Ng, and M. I. Jordan. 2003. Latent Dirichlet Allocation. *JMLR* 3, 993–1022.

[7] G. Ke et al. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *NeurIPS 30*, 3146–3154.

[8] F. Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *JMLR* 12, 2825–2830.

[9] S. Seabold and J. Perktold. 2010. Statsmodels: Econometric and Statistical Modeling with Python. In *Proc. 9th Python in Science Conf.*, 92–96.

[10] L. Breiman. 2001. Random Forests. *Machine Learning* 45, 1, 5–32.

[11] D. D. Lee and H. S. Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature* 401, 788–791.

[12] Government of India. 2023. The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023).

[13] Streamlit Inc. 2026. Streamlit Documentation. https://docs.streamlit.io/

[14] R. J. Hyndman and A. B. Koehler. 2006. Another look at measures of forecast accuracy. *International Journal of Forecasting* 22, 4, 679–688.

[15] T. Saito and M. Rehmsmeier. 2015. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE* 10, 3, e0118432.

[16] Google. 2026. YouTube Data API v3 and Google Trends. https://developers.google.com/youtube/v3; https://trends.google.com/ (accessed 3–4 Oct 2026).

[17] Group 13. 2026. Viral Product Intelligence repository. https://github.com/Ashwin-deals/viral-product-intelligence-

Scikit-learn [8], statsmodels [9] and Random Forest [10] underlie Sections 4–5; Google [16] is the data source; [17] holds all code and data.

Appendices

Appendix A: Validation records

Table *18*. Master dataset validation (processed/master_validation_report.csv)

Master dataset check	Status	Detail
rows = products x days	PASS	16140 rows = 60 products x 269 days (2026-01-08 to 2026-10-03)
no duplicate (product_id, date)	PASS	0 duplicate keys
every registry product present, nothing else	PASS	60 products; missing from data: none; not in registry: none
every product has every calendar day	PASS	269 calendar days; products with gaps: none
search_interest within 0-100	PASS	min 0.0, max 100.0, missing 0
count columns never negative	PASS	15 count columns checked; negative values: none
on_target_share within 0-1	PASS	min 0.56, max 0.96
yt_on_target_share within 0-1	PASS	min 0.48, max 1.0
zero_share_product within 0-1	PASS	min 0.0, max 0.9442
surge_label is 0 or 1	PASS	806 surge days of 16140 (4.99%)
YouTube snapshot (yt_* columns) for every product	PASS	60 of 60 products; latest snapshot date(s): 2026-10-04
date-level YouTube columns from aligned_daily	INFO	only 10 of 60 products, on 2026-10-03: snapshots taken after the Trends window ends do not join…
videos reconcile with source	PASS	3500 rows in the cleaned table = 3149 unique videos + 351 seen again in a second snapshot; 2248…
comments reconcile with source	PASS	32215 rows in the cleaned table = 29189 unique comments + 3026 seen again in a second snapshot;…
surge_label matches Unit 2 features file	PASS	16140 of 16140 rows matched by key, 0 labels differ
flagged products (kept, not dropped)	INFO	low-signal Trends: 15; low YouTube on-target share: 4 (P009, P012, P015, P028)
missing values (by design, not filled)	INFO	youtube_query_version: 16130; video_result_count: 16130; video_views_total: 16130; video_commen…

Table *19*. Method 2 leakage audit (processed/method2_leakage_audit_report.csv)

Method 2 leakage check	Status	Detail
contemporaneous_sentiment_quarantine	PASS	Contemporaneous sentiment excluded from model features; overlapping: none
contemporaneous_topic_quarantine	PASS	Contemporaneous topics excluded from model features; overlapping: none
lag_shift_integrity	PASS	Audited 107 lag index mappings for exact match Value(t-k); mismatches: 0
rolling_trend_lookahead_quarantine	PASS	Rolling window verified as strictly backwards [t-7, t-1] using shift(1).rolling(7); violations:
chronological_boundary_separation	PASS	Max train date (2026-08-11) < Min test date (2026-08-12); zero date overlap
nlp_topic_model_train_only	PASS	LDA and CountVectorizer fitted exclusively on comments with date < 2026-08-12
scaler_train_only_fit	PASS	StandardScaler fit exclusively on train partition: train feature means in [-0.01, 0.01]
point_in_time_snapshot_exclusion	PASS	Quarantined Oct-4 snapshot columns excluded from predictors; leaked: none
timezone_alignment_verification	PASS	Comment timestamps localized to Asia/Kolkata (UTC+5.5 offset verified)
target_variable_independence	PASS	Contemporaneous targets strictly excluded from predictor matrices; leaked: none
missing_signal_no_future_fill	PASS	Unobserved comment days filled with neutral 0.0 without backward filling from future days
survival_bias_audit_documented	INFO	Collection-date survival bias (SCRUM-24 warning) audited and documented: comments sampled on Oc

Appendix B: Model tuning

Table *20*. Unit 2 expanding-window CV grid (5 date folds)

Model	Settings	CV PR-AUC (mean ± SD)	Threshold	CV F1
Logistic Regression (baseline)	{C: 0.1, max_iter: 2000}	0.095 ± 0.027	0.60	0.180
Logistic Regression (baseline)	{C: 1.0, max_iter: 2000}	0.093 ± 0.026	0.60	0.172
Random Forest	{max_depth: 6, n_estimators: 300}	0.222 ± 0.016	0.56	0.273
Random Forest	{max_depth: 10, n_estimators: 300}	0.231 ± 0.025	0.45	0.293
HistGradientBoosting	{learning_rate: 0.05, max_depth: 4, max_iter: 300}	0.208 ± 0.013	0.57	0.296
HistGradientBoosting	{learning_rate: 0.1, max_depth: 6, max_iter: 300}	0.201 ± 0.019	0.59	0.261

Appendix C: Implementation and dashboard guide

Table *21*. Pipeline execution record (processed/pipeline_execution_summary.csv)

Stage	Unit	Status	Seconds	Details
task1_prep	unit1	SKIPPED	0.0	Pre-existing clean extracts found
unit2_features	unit2	SUCCESS	0.9	Engineered features & surge labels (16,140 rows, 806 surges, 30 cols)
unit1_master	unit1	SUCCESS	0.7	Master dataset validated (16,140 rows x 43 cols, 60 products)
unit3_text_corpus	unit3	SUCCESS	1.5	Text corpus extracted (29,189 comments across 60 products)
unit3_ts_prep	unit3	SUCCESS	1.4	Chronological split & stationarity diagnostics (12,540 train / 3,180 t
unit3_forecast	unit3	SUCCESS	7.6	Trained 10 forecasting models. Top model: Extra_Trees (MAE: 5.280)
unit2_method2	unit2	SUCCESS	19.9	Method 2 multimodal dataset isolated & audited (12,540 train / 3,180 t
unit3_text_mining	unit3	SUCCESS	119.3	Sentiment & topic mining completed (22,568 scored comments)
unit3_text_spotcheck	unit3	SUCCESS	0.1	Spot-check agreement evaluated (43 metric records)
unit3_text_eval	unit3	SUCCESS	1.1	Evaluated 8 sentiment methods and 10 signal lead-lag lags

Guide: run the app, choose filters in the sidebar (empty = all; they cascade and persist; *Reset filters* clears them), then switch pages. Config lives in `src/config.py`; seeds are 42; cut-off 2026-08-12.

Appendix D: Tracking, AI declaration and similarity report

Attach the Jira board screenshot (Completed column), the signed declaration (front matter, AI use in Section 8.3) and the Turnitin similarity report (**[insert %]**). Full commit history: `git log` in the repository.